

第 45 讲实验设计：Normalization 为什么有用？

一、实验目标

不用复杂模型训练，直接观察 Normalization 所针对的几个底层现象：

1. 标准化如何把一组数字变成 $\text{mean} \approx 0$ 、 $\text{std} \approx 1$ ；
2. Feature Scale 如何直接改变 Weight Gradient Scale；
3. 深层网络 Activation Scale 如何逐层漂移；
4. 输入尺度过大如何把 Sigmoid/Tanh 推入饱和区；
5. 用 PyTorch LayerNorm 预览 (B, T, D) 上的 hidden-dimension normalization。

二、实验环境

- Python 3.x
- NumPy
- PyTorch
- Matplotlib
- CPU 即可

三、实验 1：手工标准化与 shift / scale

输入：

$$x = [2, 4, 6, 8]$$

记录 mean、variance、std 与标准化结果。再构造：

$$x' = 10x + 100$$

验证二者标准化结果几乎一致。

假设：整体正比例放大和平移会被标准化消除。

四、实验 2：Feature Scale 与 Gradient Scale

构造：

$$x = [1, 100, 10000], \quad w = 0, \quad y = 1$$

对于：

$$L = (w^T x - y)^2$$

计算：

$$\frac{\partial L}{\partial w}$$

然后把三个 feature 缩放到相同数量级，再比较 Gradient。

观测指标：最大/最小 Gradient 绝对值比例。

五、实验 3：20 层 Activation Scale 漂移

固定同一组 Weight，比较：

1. Linear + ReLU;
2. Linear + 概念性按最后一维标准化 + ReLU。

记录每一层 Activation mean/std。

注意：这里的标准化只是原理演示，不等于正式 LayerNorm 实现教程。

六、实验 4：Activation Saturation

让：

$$z \sim \mathcal{N}(0, s^2)$$

逐渐增大 s ，统计：

$$\sigma'(z) < 0.01$$

和：

$$\tanh'(z) < 0.01$$

的样本比例。

预期：输入尺度越大，饱和比例越高。

七、实验 5：LayerNorm 功能预览

构造：

$$X.shape = (2, 2, 4)$$

使用：

```
1 ln = torch.nn.LayerNorm(4, elementwise_affine=False)
2 Y = ln(X)
```

检查：

- Shape 是否改变；
- 每个 token hidden mean 是否约 0；
- 每个 token hidden std 是否约 1；
- [1, 2, 3, 4] 与 [10, 20, 30, 40] 是否得到相同标准化形状。

八、核心结果

本次实际运行：

- 原始梯度尺度最大/最小比例：10000；缩放后变为同量级；
- 无标准化 20 层 Activation Std 最终约 17.445；
- 概念性标准化后 20 层 Activation Std 约 0.566；
- 输入 Std=10 时，Sigmoid 饱和比例约 0.644，Tanh 约 0.762；
- LayerNorm 预览中每个 token 的 hidden mean 约 0、population std 约 1。

九、结论边界

这些实验支持：Normalization 可以显著改变数值尺度、激活分布与优化条件。但不能从这些小实验推出“某种 Normalization 一定让所有模型泛化更好”或“Normalization 的唯一机制就是控制均值方差”。具体方法还需要结合 architecture、axis、train/eval 统计方式和任务验证。